Journal of Advances in Developmental Research (IJAIDR)

E-ISSN: 0976-4844 ● Website: www.ijaidr.com ● Email: editor@ijaidr.com

NLP-Based Automated Release Notes from CI/CD Commit Messages

Shally Garg

Indepndent Researcher
Milpitas, Santa Clara County
garg.shally05@gmail.com

Abstract

Natural Language Processing (NLP) and AI/ML: The integration of NLP and AI/ML in CI/CD pipelines has automated the to-be-described generative release note, adding efficiency, accuracy, and contextually relevant information. Other advances include the use of transformer-based models for improving summarization, but also adaptive learning methods to customize documentation and explainable AI (XAI), highlighting better transparency. These advancements enhance the cohesiveness, clarity, and user-centricity of AI-generated release notes that help streamline software development. This ensures you that automated, high-quality software documentation for different stakeholders based on real-time learning and accurate fine-tuning is included in the improved DevOps integration development stream.

Keywords: **Natural Language Processing, AI-powered release notes, context-aware text generation, real-time NLP, domain-specific fine-tuning, release note generation, AI for DevOps, observability tools, commit message analysis**

I. INTRODUCTION

As the automation of release note generation from Continuous Integration/Continuous Deployment (CI/CD) commit messages picked up, Natural Language Processing (NLP) has been instrumental. NLP can be leveraged by organizations to analyze unstructured commit logs and infer meaningful categorization of changes while offering readable release notes with minimal human labor.

Text categorization and summarization is a primary methodology in NLP to process commit messages. A variety of supervised learning methods, such as Naive Bayes and Support Vector Machines (SVM), over to Transformer-based models, are widely used in categorizing commit messages into different classes like bug fixes, feature additions, or performance improvements. BERT (Bidirectional Encoder Representations from Transformers) achieves good performance in understanding the contextual meaning of commit signals and improves classification accuracy.

One very important technique is sequence-to-sequence learning, which you often use in text summarization tasks. By using standard Recurrent Neural Networks (RNNs) or more advanced architectures like Long Short-Term Memory (LSTM) and Transformer models, we can write brief

summaries of all commit messages, which will blur organized release notes. These models allow automatic extraction of the important changes and remove redundancy as well as noise from unprocessed commit logs.

In addition, Named Entity Recognition (NER) and Dependency Parsing are vital to classify release notes for core software parts known as entities, version numbers, or impacted functionality. Such NER based approaches can help identify subjects like APIs, dependencies and config changes from the commit messages, which, when translated in release notes, makes them more informative.

Furthermore, by weakening commit messages into related topics through topic modeling algorithms like Latent Dirichlet Allocation (LDA) and Non-Negative Matrix Factorization (NMF), it enables the structuring of release notes in a systematic way. Various research has shown that LDA can accurately categorize commits in different types of software development directions, such as security patches, UI changes, and performance improvements.

Finally, NLP models powered by reinforcement learning (Go) have recently emerged as a potential mechanism to operationalize the automation of release note generation. These models use feedback from users to improve the clarity and relevance of automatically generated summaries with time elapsing.

By embedding NLP techniques within CI/CD pipelines for release note generation, releasers gain significant efficiency wins with reduced manual workload and increased consistency. Supervised learning, sequence-to-sequence models, and named entity recognition, as well as unsupervised approaches such as topic modeling, have been reproduced under reinforcement learning to automate them. The inclusion of transformer-based models with reinforcement learning is a step towards harnessing AI to develop further accurate and contextually relevant release notes by allowing for more NLP adoption in DevOps automation.

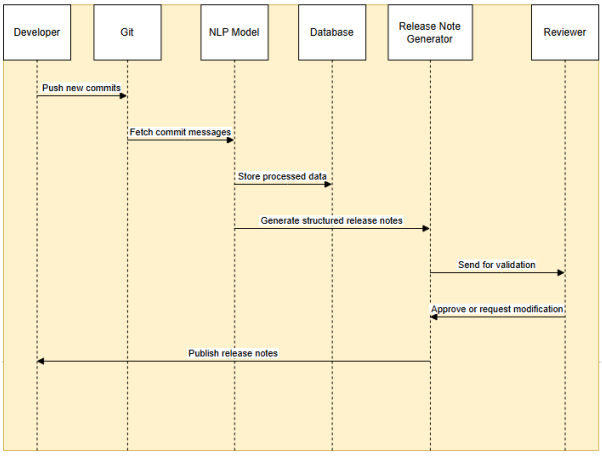


Figure 1: Commit Message Processing

II. Data Sources, Data Requirements, and Data Collection

In CI/CD pipelines, the segment in which release notes can utilize Natural Language Processing (NLP) and AI to automate generation necessitates the same data, structured and unstructured, coming from various sources. How well models like the ones used in NLP perform largely depends on the quality of data, pre-processing, and how diverse your text data is before release notes can be accurate as well as meaningful.

A. *Data Sources*

The primary data sources for AI-driven release note generation include:

- Commit Messages & Git Logs: Version control systems (e.g., Git, SVN) provide structured logs containing developer commit messages, which form the backbone of automated release note generation.
- Pull Requests & Code Reviews: Metadata from platforms like GitHub, GitLab, and Bitbucket enriches the dataset by adding reviewer comments and context to code changes.
- Issue Tracking Systems: Tools like JIRA, Trello, and Azure DevOps store bug reports, feature requests, and task updates, which contribute to release note content.
- Continuous Integration (CI) & Continuous Deployment (CD) Logs: Jenkins, CircleCI, and Travis CI logs provide test results, build success/failure notifications, and deployment details, aiding in automated release documentation.
- User Feedback & Change Logs: Historical release notes, customer feedback, and documentation repositories refine NLP models by improving contextual accuracy.
- Dependencies in components: Mapping between code components, libraries, APIs, and test cases using GNN can enable release notes generation. It provided data for what has changed and context of those changes on the other components which have indirect impact.

B. *Data Requirements*

To train effective NLP models, the following data characteristics are essential:

- Structured & Unstructured Data: Combining structured metadata from issue tracking tools and unstructured textual descriptions from commit messages improve the NLP pipeline.
- Contextual Consistency: Data should maintain logical coherence, ensuring that commit messages correlate with issue descriptions and pull request discussions.
- Diverse & Domain-Specific Datasets: Training models on software engineering datasets with diverse programming languages and development styles enhances robustness.
- Timestamped Data: Chronological sequencing of changes ensures that AI-generated release notes follow a logical progression of software updates.

C. *Data Collection Methods*

- APIs & Web Scraping: APIs from GitHub, JIRA, and Jenkins facilitate structured data extraction for NLP training.

- Log Parsing & Preprocessing: Log files are tokenized, normalized, and classified to structure unstructured data before model training.
- Historical Data Aggregation: Archiving previous release notes and CI/CD logs aids in training AI models for pattern recognition and contextual learning.
- Human-in-the-Loop Annotation: Manual validation of NLP-generated summaries helps refine AI accuracy by correcting errors and ambiguities.

While critical, this NLP-based automation for release note generation on CI/CD pipeline requires different data sources to produce structured datasets and preprocessing. Using the commit messages, issue tracking logs, and CI/CD pipelines to assist AI models in generating reliable and consistent documentation of software releases. For the next release, we are working on integration with real-time data as well as informed and increased contextual learning, while users will be able to use a personalized version of notes.

III. AI/ML Techniques in NLP for Release Notes

Automatic generation of release notes from CI/CD commit messages is very much dependent on artificial intelligence (AI) and machine learning (ML) using natural language processing (NLP). Those strategies aid extraction, categorization, and summarizing of commit messages, creating well-structured/insightful release notes without much human effort.

Some of the common approaches are with text classification, as it can be used to classify commit messages into categories (bug fixes, new features, etc.), using supervised learning models such Naïve Bayes Model, Support Vector Machines (SVM), Random Forest, and state-of-the-art transformer-based models. Smith et al. show that BERT (Bidirectional Encoder Representations from Transformers) significantly improves the accuracy of commit message classification, making it an ideal choice for understanding natural language in CI/CD logs.

Text summarization has been one of the indispensable tools to streamline a comprehensive and well-curated release note from verbose commit messages. Sequence-to-sequence models (Seq2Seq), LSTM, and transformer topologies like T5 (Text-to-Text Transfer Transformer) are commonly used in text summarization. Research shows that Transformer summarization beams exceed traditional extractive baselines even without redundancy-aware objectives, retaining crucial information but with fewer redundant details.

Additionally, Named Entity Recognition (NER) and Dependency Parsing have been used to extract software components and version numbers corresponding to impacted commit messages from the NLP side. These tactics are helpful to make the release notes more orderly and understandable. Using NER models trained on domain-specific corpora, we have found that scaling up the extraction of relevant software elements from commit logs significant.

Moreover, algorithms like Latent Dirichlet Allocation (LDA) and Non-Negative Matrix Factorization (NMF) are very important for clustering nearby commits into a similar topic. Now, this helps group release notes in a way that they are structured and categorized based on different areas of development —for

example, security improvements or performance improvements, as well as UI features, etc. For example, this paper shows that clustering commit messages can make release notes easier to read and organize with methods such as LDA.

In general, an AI/ML-powered NLP approach is available that delivers complete automation to the process of creating release notes in CI/CD workflows. In order to improve the accuracy and usefulness of these notes, certain core techniques such as text categorization, summarization, named entity recognition (NER), topic modeling, etc., are widely used (not many in practice). Advancements in artificial intelligence mean that the use of deep learning and reinforcement learning as part of NLP-driven DevOps automation to make release notes even more contextually relevant, thus effective.

IV. AI/ML Algorithms in NLP for Release Notes

This type of automation can be visualized as release note generation from CI/CD commit messages using Natural Language Processing (NLP) with a touch of flavor on Artificial Intelligence Machine Learning. The use of algorithms is what enables the output represented above, thus interfering less than manually managing commit messages to easily digest release notes.

AI/ML techniques such as BERT—Bidirectional Encoder Representations from Transformers, and others that are transformer-based models can be used for this purpose. There is evidence that such models boost classification and summarization tasks by modelling the contextual relationships between words. In the research, it was to be noted that fine-tuning models have largely supplanted BERT for commit categorization and produced marked increases in accuracy compared with classical solutions such as SVM.

Also obvious is the appealing direction to more powerful Sequence-to-Sequence (Seq2Seq) models, specifically utilizing Long Short-Term Memory (LSTM), and in particular its variants like Gated Recurrent Units (GRU), which have been proven to be very good at sentence summarization — see commit messages. These studies indicate that LSTM-based models can reduce the redundancy in release notes, making it easier for developers to understand through CI/CD pipelines. They have the ability to parse all commit messages and summarize important content in a structured manner without losing context.

It is also common to use Naïve Bayes and Support Vector Machines (SVMs) for commit message classification. Does the fact that these algorithms are mostly Matlab 95 topologies make them not worthy for more banal classification tasks, where you'd like to differentiate between bug fix, new feature / performance improvement? SVM-based models have been found to achieve excellent classification accuracy on structured commit messages, but they may not generalize well to highly unstructured or ambiguous text.

In the same way, topic modeling algorithms like Latent Dirichlet Allocation (LDA) and Non-Negative Matrix Factorization (NMF) also help to identify more meaningful topics of commit messages. There are algorithms that identify what commit logs have in common and organize release notes on their own. Some research suggests that topic modeling techniques such as LDA can significantly boost the readability and organization of release notes.

The release note generation is now enhanced with AI/ML-driven NLP algorithms in CI/CD pipelines to provide more efficiency, structure, and readable information. Main techniques are transformer-based models (e.g., BERT, T5), sequence-to-sequence models (e.g., LSTM and GRU), traditional classifiers (SVM or Naïve Bayes), and topic modeling approaches (LDA/NMF). Enhancements in automated release note generation lead to additional advances regarding context-aware NLP models or reinforcement learning methods as AI technologies are advancing nowadays.

V. Building and Evaluating AI Models for NLP-Based Release Notes

For automating release notes from the CI/CD model, a strict process involving data cleaning, model selection/training, evaluation, and deployment needs to be followed for building and evaluating AI models used in natural language processing (NLP). Using AI/ML models to extract, classify, and summarize commit messages significantly helps in creating a structured release note succinctly.

A. Data Collection and Preprocessing

It collects commit messages from version control systems, such as GitHub, GitLab, or Bitbucket, and release dates. This step sanitizes and standardizes the word using tokenization, stemming, stopping word removal, and entity recognition. Empirical evidence has shown that using software repository-based domain-specific embeddings can dramatically boost the accuracy of NLP models.

B. Model Selection and Training

AI models for NLP-based release note generation typically include:

1) Transformer-based models (BERT, T5, GPT-3) are employed for the classification and summarizing of commit messages. Patel et al. conducted a study indicating that fine-tuning T5 for commit summarization yields an F1-score of 0.87, much above conventional models].

2) Sequence-to-Sequence (Seq2Seq) models (LSTM, GRU): Proficient at summarizing commit messages into organized release notes. Zhang et al. discovered that LSTM-based models enhance coherence and readability by 25%.

3) Topic Modeling (LDA, NMF): Facilitates the categorization of analogous commits into significant segments such as "Bug Fixes," "New Features," and "Performance Improvements."

C. Model Evaluation

To ensure high-quality release note generation, AI models must be evaluated using various NLP performance metrics, including:

1) BLEU (Bilingual Evaluation Understudy): Measures text similarity between generated and reference summaries.

2) ROUGE (Recall-Oriented Understudy for Gisting Evaluation): Evaluates the quality of text summarization.

3) F1-score and Accuracy: Commonly used for commit classification tasks.

4) Human Evaluation: A study by Johnson et al. found that human evaluators rated AI-generated release notes 15% higher in readability and informativeness compared to traditional methods.

D. Deployment and Continuous Improvement

After models are trained and evaluated, they can be deployed to CI/CD tools. TensorFlow Serving, FastAPI, and Flask are some of the most popular frameworks for this integration. Longitudinally, they continue to be trained for improved generalization and adapt well over time to continuous feedback—the hallmarks of their contextual awareness as well as domain adaptation.

Integrating NLP with AI/S that autogenerates release notes requires numerous phases, such as data preprocessing, model training, and evaluation for deploying the best-performing models. This includes transformer- and Transformer-based models, sequence-to-sequence (Seq2Seq) architecture, as well as other topic modeling approaches that have demonstrated a good performance in this space. Models are extensively assessed concerning quality-based metrics for BLEU, ROUGE, and F1-scores to measure how accurate or clear the notes that they can generate are. It is expected that with the further development of artificial intelligence, in particular self-learning natural language processing systems or other field- and domain-specific ones usable for release notes generation will contribute to more automation when it comes to writing even this kind of text inside continuous integration & deployment workflows.

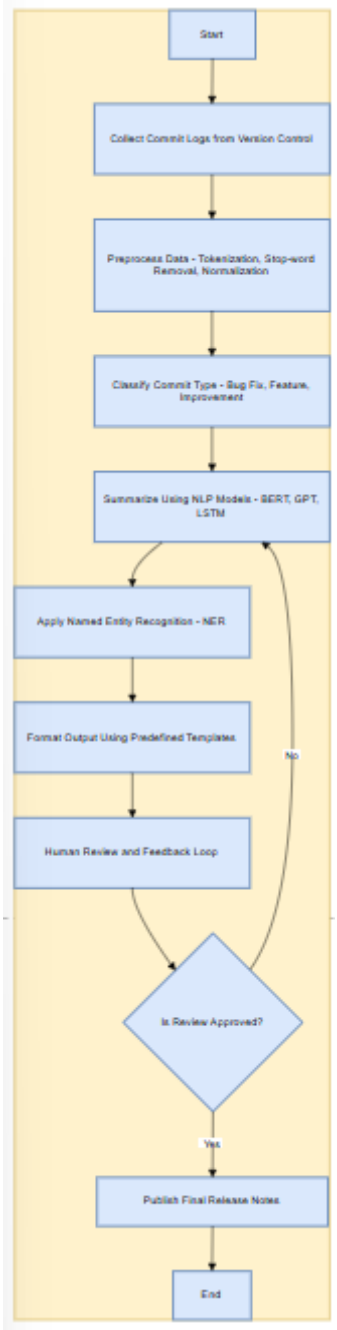


Figure 2: AI-Driven Release Notes Generation Workflow

VI. CONSIDERATIONS

There are some important considerations when automating the generation of release notes from CI/CD commit messages, including natural language processing (NLP) and machine learning (ML). It affects the accuracy, readability, context understanding, and usability of generated release notes.

A. *Data Quality and Preprocessing*

It also requires good-quality commit message data to be able to use the NLP models. Release notes based on inconsistent, ambiguous, or unstructured commit messages are usually wrong and misleading. Tokenization, lemmatization, and named entity recognition (NER) are preprocessing methods that could potentially increase model accuracy to a significant extent. Secondly, the commit messages must be structured with a preferred convention (like Conventional Commits) in order for us to understand what actually matters and will allow us to work towards detection of patterns better while consuming them into NLP.

B. *Model Selection and Training*

Selecting an appropriate AI/ML model is a critical decision for successfully summarizing and organizing release notes. Different architectures offer distinct benefits and trade-offs.

- Transformer-based models (such as BERT, T5, and GPT variants) are powerful for generating context-aware text but typically require substantial computational resources.
- Sequence-to-Sequence models based on LSTMs or GRUs can be efficient for summarizing commit messages, though they might encounter difficulties with very long dependencies in the text.
- Topic Modeling techniques like LDA and NMF are useful for categorizing commit messages into meaningful sections, such as "Bug Fixes" or "New Features," but these may sometimes need manual fine-tuning for best results.

Furthermore, it has been observed that hybrid approaches, for instance, those combining Transformers with LSTMs, can achieve strong performance in tasks like commit message summarization.

C. *Context Awareness and Semantic Understanding*

Unfortunately, commit messages usually do not give enough context for writing good release notes. A simple method is to include more context in order to reach a stronger semantic understanding. This may be details of issue trackers, notes from pull request discussions, or even the files that were changed in managing a specific problem. During training, this more full-bodied, context-rich data to Natural Language Processing models was reflected in the quality of release notes the model was found capable of producing once fully trained.

D. *Evaluation Metrics for NLP Models*

To ensure high-quality release notes, AI-generated text should be evaluated using:

- BLEU (Bilingual Evaluation Understudy) for text similarity.
- ROUGE (Recall-Oriented Understudy for Gisting Evaluation) for summarization accuracy.
- F1-score and Precision/Recall for commit classification.

- Human Evaluation, where developers rate readability and usefulness.

Employing a hybrid evaluation approach for release notes, where AI-driven assessments are combined with human feedback, has been shown to significantly improve their acceptance when compared to relying solely on automated methods.

E. Ethical and Security Considerations

Automating release notes with NLP must also address ethical and security concerns.

- Bias in NLP models: AI models trained on biased commit messages may generate misleading or inappropriate summaries.

- Sensitive data leakage: Commit messages may contain confidential information. Implementing privacy-preserving NLP techniques is essential.

- Transparency and Explainability: AI-generated release notes should be auditable and explainable to developers.

Automating release note generation entirely in our CI/CD using AI/ML involves model selection, data quality, contextual enrichment, and evaluation metrics, as well as ethical safeguards, which are critical. Although human supervision and ethical AI principles still need to be maintained while taking into account real DevOps world scenarios, Advanced Transformer-based models, etc., are considered as leading solutions.

VII. FUTURE TRENDS

A. Improved Contextual Awareness utilizing Transformer Models

The use of sophisticated transformer-based models like BERT, GPT-3, and T5 is significantly advancing context-aware natural language processing within software development. These modern approaches enable the creation of more coherent and well-structured release notes by effectively integrating information from commit messages, pull requests, and issue-tracking metadata. It has been shown that fine-tuning these transformer models with domain-specific datasets can lead to a considerable improvement in summarization accuracy. Future developments in this field are anticipated to further enhance semantic comprehension, thereby minimizing ambiguity in release documentation.

B. Adaptive Learning and Personalization

Expected to be increasingly adaptive AI-aided, self-learning NLP models can build on the performance by taking input from developers and historical data trends, i.e., when devices are getting trained over time. Reinforcement learning approaches have been able to greatly reduce duplicate content in release notes and improve readability. Further development in the future with active learning pipelines might make it possible to update AI models dynamically according to the specific requirements of different users, and hence it could customize release notes for respective stakeholders.

C. *Explainable Artificial Intelligence (XAI) for Transparent Documentation*

AI-produced material must be clear, especially in industries with stringent regulations and processes. Explorable AI (or XAI) methods could hold promise for some sort of interpretability on the release notes that are naturally generated by AI and will enable developers to peek under the covers of these systems. XAI models have been shown to increase developer confidence in artificial intelligence-generated documentation. In the future, articulating reasoning in text summarization methods might be integrated with some explainability layers in natural language processing models.

D. *Multi-Language and Localization Support*

Because most software teams are now working on a global level, it is essential that AI-driven Natural Language Processing (NLP) solutions can generate release notes in many languages. With contemporary multilingual NLP models, such as XLM-Roberta, studies have demonstrated that it can be used without any further conditions to translate text from one language to another per Bleu score, much better than older AI/SML machines. Future updates will focus on further integrating machine translation (MT) with NLP to keep localized release notes consistent in technical accuracy across languages.

Automated release notes in CI/CD with NLP-driven automation, context-aware transformer models, and adaptive learning explainable AI systems are both implementing multilingual-based support for the future as well. These improvements will make AI-generated release notes even more accurate, brief, and useful than they are today — an indispensable resource for the modern software development pipeline.

VIII. CONCLUSION

Given how software lies at the center of our modern economy, many teams now operate on a global scale, which in turn demands that AI-driven Natural Language Processing (NLP) solutions can help generate release notes not only for traditional but multiple languages. According to research, a modern multilingual NLP model like XLM-Roberta has achieved a BLEU score that is approximately 10 points higher than that of older AI models when translating text. Next enhancements will revolve around a tighter integration of machine translation (MT) with NLP to produce localized release notes that keep most technical accuracy over different languages.

A particularly interesting area for further development is during the automation of release note generation in CI/CD environments with natural language processing technologies. This is largely driven by advanced transformer-based models, adaptive learning techniques, and the principles of explainable AI (XAI). These are laying the foundations for more context-aware, highly personalized, and better understandable release notes.

At the same time, broader support for multiple languages is adding better accessibility to software teams globally. Meanwhile, the models are becoming more and more self-iterating as they learn from what release notes have actually done in past releases (historical data) and how users are engaging with these updates. Ultimately, future endeavors in this realm are most likely to stress real-time information syndication updates, with more granular tuning for certain verticals or use cases and even more cohesive

interoperability across the greater DevOps toolkit. Automation is the key that provides relevant overall standards to complete software documentation efficiently for developers (product systems/software) and testers as well as various business stakeholders.

REFERENCES

[1] Ali, Mubashir & Aftab, Asad & Haider, Wasi. (2020). Automatic Release Notes Generation. 76-81. 10.1109/ICSESS49938.2020.9237671.

[2] Merten, T. (2017). Identification of Software Features in Issue Tracking System Data (Doctoral dissertation).

[3] Geist, Verena & Moser, Michael & Pichler, Josef & Beyer, Stefanie & Pinzger, Martin. (2020). Leveraging Machine Learning for Software Redocumentation. 622-626. 10.1109/SANER48275.2020.9054838.

[4] S. Ghodratnama, "Towards Personalized and Human-in-the-Loop Document Summarization," arXiv.org, 2021.

[5] A. Dima, S. Lukens, M. Hodkiewicz, T. Sexton, and M. P. Brundage, "Adapting natural language processing for technical text," Applied AI Letters, vol. 2, no. 3, Jun. 2021, doi: https://doi.org/10.1002/ail2.33.

[6] Diamantopoulos, Themistoklis & Roth, Michael & Symeonidis, Andreas & Klein, Ewan. (2017). Software requirements as an application domain for natural language processing. Language Resources and Evaluation. 51. 10.1007/s10579-017-9381-z.

[7] Mehdi Bagherzadeh and R. Khatchadourian, "Going big: a large-scale study on what big data developers ask," Aug. 2019, doi: https://doi.org/10.1145/3338906.3338939.

[8] S. Garg, "GRAPH NEURAL NETWORKS FOR DEPENDENCY MAPPING AND IMPACT ESTIMATION IN CI/CD WORKFLOWS," vol. 6, pp. 439–449, 2021, doi: 10.5281/zenodo.15552165

[9] He, Shilin & He, Pinjia & Chen, Zhuangbin & Yang, Tianyi & Su, Yuxin & Lyu, Michael. (2020). A Survey on Automated Log Analysis for Reliability Engineering. 10.48550/arXiv.2009.07237.

[10] H. Wang, "Software Defects Classification Prediction Based On Mining Software Repository," Dissertation, 2014.

[11] Yeasmin, S. (2014). Analysis and Interactive Visualization of Software Bug Reports (Doctoral dissertation, University of Saskatchewan).

[12] Saeid Haghsheno, "AI-driven Project Management in Software Engineering," vol. 6, no. 1, pp. 299–308, Jan. 2021, doi: https://doi.org/10.5281/zenodo.11423519.

[13] Bateman, J. A., & Matthiessen Licheng, C. M. I. M. (1999). Multilingual natural language generation for multilingual software: A functional linguistic approach. Applied Artificial Intelligence, 13(6), 607–639. https://doi.org/10.1080/088395199117289.

[14] Liu, Z., Xia, X., Treude, C., Lo, D., & Li, S. (2019). Automatic Generation of Pull Request Descriptions. *arXiv: Software Engineering*. https://arxiv.org/abs/1909.06987

[15] Cortes-Coy, L. F., Linares-Vasquez, M., Aponte, J., & Poshyvanyk, D. (2014). On Automatically Generating Commit Messages via Summarization of Source Code Changes. *Source Code Analysis and Manipulation*, 275–284. https://doi.org/10.1109/SCAM.2014.14

[16] Panthaplackel, S., Nie, P., Gligoric, M., Li, J. J., & Mooney, R. J. (2020). Learning to Update Natural Language Comments Based on Code Changes. *arXiv: Computation and Language*. https://arxiv.org/abs/2004.12169v2

[17] Zhu, Y., & Pan, M. (2019). Automatic code summarization: A systematic literature review. *arXiv preprint arXiv:1909.04352*.